

G. Yashwanth
192524275

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA12

COURSE OUTCOME COVERED

CO1: Analyze the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)
Assessment Tool Weightage: 40%

QUESTIONS:5

Total Marks:25

**ANNEXURE A
APPLICATION BASED QUESTIONS**

18/18

Code of Conduct:

*I **G.YASHWANTH (192524275)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:G.YASHWANTH

ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.
2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch-decode-execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.
3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes

of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.
5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)	
Q1	3	5	3	4	2	17
Q2	3	4	3	3	1.5	14.5
Q3	4	5	4	2	1.5	16.5
Q4	2	4	4	2	2	14
Q5	3	6	3	3	2	16

78

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements ⁴	Good understanding with minor gaps ³	Basic understanding ; some requirements unclear ²	Poor understanding or misinterpretation of the problem ¹
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems ⁶	Applies relevant concepts correctly with minor errors ⁵	Applies basic concepts with limited accuracy ⁴	Fails to apply appropriate concepts ¹⁻²
Problem-Solving Approach	20	Logical, well-structured, and efficient approach ⁴	Mostly logical approach with minor gaps ³	Basic approach with some inconsistencies ²	Illogical or unclear approach ¹
Accuracy of Solution	20	Completely correct solution with proper justification ⁴	Mostly correct with minor errors ³	Partially correct solution ²	Incorrect or incomplete solution ¹

Clarity	10	2 Clear, well-organized, and properly explained solution	1.5 Understandable with minor clarity issues	1 Limited clarity and explanation	0 Poorly presented and difficult to understand
---------	----	--	--	-----------------------------------	--

ANSWERS

1. Smart City Surveillance System Answer:

In a smart city surveillance system, multiple cameras continuously send video data to the CPU through memory and I/O devices. During peak hours, the CPU receives a huge amount of data, causing delays in processing.

Interaction between CPU, Memory, and I/O

- I/O Devices: Cameras capture video and transfer it to memory.
- Memory (RAM): Temporarily stores video frames.
- CPU: Fetches data from memory, processes it, and detects threats.

When many cameras send data simultaneously:

- Memory bandwidth becomes busy.
- CPU waits for data from memory.
- I/O devices compete for the system bus.
- This creates bottlenecks and increases response time.

Single-Bus vs Multi-Bus Structure:

Single-Bus System

- One bus connects CPU, memory, and I/O.
- Only one data transfer occurs at a time.
- Causes traffic congestion.
- Lower performance.

Multi-Bus System

- Separate buses for CPU, memory, and I/O.
- Multiple transfers occur simultaneously.
- Reduces waiting time.
- Improves throughput and system speed.

CPU follows:

1. Fetch
2. Decode
3. Execute
4. Memory Access
5. Write Back

Performance can be improved using:

- Instruction pipelining
- Cache memory
- DMA (Direct Memory Access)
- Branch prediction
- Multi-core processors

Conclusion

Using a multi-bus architecture, cache memory, DMA, and pipelining reduces CPU waiting time and significantly improves surveillance system performance.

2. Real-Time Stock Trading Application Answer:

A stock trading application executes millions of instructions every second. High memory access and frequent branch instructions increase the **Cycles Per Instruction (CPI)**, causing transaction delays.

Fetch-Decode-Execute Cycle

1. Fetch instruction from memory.
2. Decode the instruction.
3. Execute the operation.
4. Access memory if required.
5. Store the result.

Frequent memory access slows the fetch stage, while branch instructions interrupt the instruction pipeline.

Reasons for High CPI

- Cache misses
- Memory latency
- Frequent branch instructions
- Pipeline stalls
- Data hazards

Methods to Reduce CPI

- Larger and faster cache memory
 - Branch prediction
 - Instruction pipelining
 - Out-of-order execution
 - Superscalar architecture
 - Prefetching instructions
- CPU Performance Formula:**

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

Reducing CPI directly decreases execution time.

Conclusion

Efficient cache design, branch prediction, pipelining, and advanced CPU architecture reduce CPI and improve transaction speed.

3. Embedded System using 8085 Microprocessor Answer:

An 8085-based temperature controller receives sensor data and controls actuators. Incorrect addressing modes may cause wrong temperature readings.

Addressing Modes in 8085

1. Immediate Addressing
 - Data is part of the instruction. ◦ Example: `MVI A,25H`
2. Register Addressing
 - Data stored in registers. ◦ Example: `MOV A,B`
3. Direct Addressing
 - Memory address specified directly. ◦ Example: `LDA 2500H`
4. Register Indirect Addressing
 - Register pair stores memory address. ◦ Example: `MOV A,M`
5. Implied Addressing
 - Operand is implied. ◦ Example: `CMA`

Causes of Incorrect Temperature Readings

- Wrong memory address
- Incorrect register selection
- Sensor data overwritten

- Improper register pair usage
 - Incorrect instruction sequence
- Effective Use of 8085 Instruction Set •** Use correct addressing mode.
- Validate sensor data.
 - Store data in dedicated memory locations.
 - Use comparison instructions (CMP).
 - Use conditional jumps (JZ, JC, JNZ) for decision making.

Conclusion
Proper addressing modes and correct instruction usage ensure accurate sensor readings and reliable temperature control.

4. 8086 Segmented Memory System Answer:

The 8086 microprocessor divides memory into segments for efficient program execution.

Types of Segments

1. **Code Segment (CS)** – Stores program instructions.
2. **Data Segment (DS)** – Stores variables and data.
3. **Stack Segment (SS)** – Stores function calls and local data.
4. **Extra Segment (ES)** – Used for additional data storage.

Working of Segmentation

Physical Address = Segment Address $\times$ 10H + Offset Example:

- Segment = 2000H
- Offset = 3000H

Physical Address = **23000H**

Problems Due to Improper Segmentation

- Incorrect memory access
- Stack overflow
- Program crashes
- Invalid instruction execution
- Data corruption

Managing Instruction Execution •

Load correct segment registers.

- Initialize stack properly.
- Use correct addressing modes.
- Maintain stack using PUSH and POP.
- Use CALL and RET correctly.

Conclusion

Proper segment management prevents memory errors and ensures reliable program execution.

5. Number System Conversion in Data Communication

Answer:

Communication systems often represent packet data in hexadecimal because it is compact and easy to read. Internally, the CPU processes data in binary.

Importance of Hexadecimal–Binary Conversion

- Easier debugging
- Compact representation
- Accurate instruction decoding
- Simplifies memory addressing
- Efficient data transmission

Effects of Conversion Errors

- Incorrect instruction execution
- Wrong memory address

- Corrupted data
- Communication failure
- Packet loss

Efficient CPU Design

- Error detection mechanisms
- Cache memory
- Fast instruction decoding
- Efficient ALU operations
- High-speed buses

Role of Benchmarking

Benchmarking measures system performance by evaluating:

- Execution time
- Throughput
- CPI
- Memory access time
- CPU utilization

It helps identify bottlenecks and improve system performance.

Conclusion

Accurate hexadecimal-to-binary conversion, efficient CPU architecture, and benchmarking ensure reliable data processing and communication.

Handwritten signature

Handwritten flourish